Feature Monitor

Repository setup report

Prepared October 8, 2026 · Scope: initial project skeleton

What this repository is for

Feature Monitor is planned as a system that watches a deployed language model for changes in its behavior. The eventual system is intended to inspect internal model activations, encode them into sparse autoencoder (SAE) features, count frequently active features, and alert when those counts shift from a baseline. A prompt-embedding baseline is also planned for comparison.

Important: this work established the repository foundation only. The model monitoring, feature extraction, alerting, API, storage, scheduler, and dashboard are not implemented yet.

Project foundation

Item	What was set up
Project name	`feature-monitor`, using Python package metadata in `pyproject.toml`.
Python	Python 3.12 or newer is declared; uv created its project environment with Python 3.12.14.
Package manager	uv is the project's manager, consistent with `CLAUDE.md`. The generated `uv.lock` records resolved dependency versions.
Runtime dependencies	None yet. No ML libraries were installed or added.
Development dependencies	pytest for tests and Ruff for linting/formatting. Resolved versions: pytest 9.1.1 and Ruff 0.16.10.

uv creates and uses a local `.venv` directory for this project. That directory is ignored by Git and should not be committed.

Folders and files

Path	Purpose at this stage
`backend/`	Python package reserved for the API and storage layer; currently only has its package marker.
`monitor/`	Python package reserved for feature extraction and drift detection; currently only has its package marker.
`traffic/`	Python package reserved for traffic generation and scenario configurations; currently only has its package marker.
`eval/`	Python package reserved for experiments and results; currently only has its package marker.
`frontend/`	Reserved for the plain HTML, CSS, and JavaScript dashboard. A keep-file ensures the empty folder is present.
`docs/`	Documentation folder. `log.md` records the setup work and remaining scope.
`tests/`	Contains a smoke test and mirrored subfolders for backend, monitor, traffic, eval, and frontend. Python test folders have package markers; the frontend test folder has a keep-file.

An `__init__.py` file marks a folder as a Python package. These empty files make the planned Python modules importable, but they do not contain application behavior.

Quality checks and commands

Ruff is configured in `pyproject.toml` with a 100-character line limit and the `I` lint rule, which checks import sorting.

The root `Makefile` provides three shortcuts:

Command	Action
`make test`	Runs `uv run pytest`.
`make lint`	Runs `uv run ruff check .`.
`make format`	

Runs `uv run ruff format .`.
`.gitignore` excludes Python bytecode and test/lint caches, uv environments/caches, common macOS metadata, model/framework cache directories, and SQLite database files.

Verification

Check
Observed result
`uv run python --version`
Python 3.12.14.
`uv run pytest`
Passed: 1 smoke test.
`uv run ruff check .`
Passed: "All checks passed!"
The smoke test is intentionally trivial: it verifies that pytest discovers and runs a test. It does not test model behavior because no application code exists yet.

Environment note: the shell had an active `VIRTUAL_ENV` pointing to a different project. uv warned that it did not match this repository and ignored it, then used this project's `.venv`. The tests and lint check completed successfully with the project environment.

What remains

The next implementation milestones can add the actual monitoring behavior, tests, API, persistence, scheduler, and interface. ML packages should be added only when their milestone needs them, and their installed versions and APIs should be checked as requested in `CLAUDE.md`.

This report describes the repository state after the initial skeleton setup; it does not claim that planned system components are already working.